

CHAPTER 4

SYSTEM DEVELOPMENT, IMPLEMENTATION, TESTING, EVALUATION, RESULTS, AND DISCUSSION

4.1 Introduction

This chapter presents the development, implementation, testing, evaluation, results, and discussion of the LEXSHIELD system. LEXSHIELD is a web-based legal services and case management system developed to support the work of administrators, lawyers, and clients in a structured digital environment. The system provides organized access to lawyer profiles, client records, case files, appointments, messages, manual payments, billing records, and system settings.

The purpose of this chapter is to describe how the completed project was built and how each major module contributes to the objectives of the study. It also explains the technical requirements, system architecture, database design, security controls, testing activities, implementation process, and observed results. The discussion focuses on the actual project files and functional scope of the system.

The system was developed for a local web environment using PHP, MySQL, JavaScript, CSS, and XAMPP. It includes separate dashboards for the three main user roles. Administrators manage lawyers, clients, inquiries, payments, messages, and settings. Lawyers view case files, appointments, profile information, client-related records, and secure messages. Clients register, log in, browse lawyers, manage profiles, submit manual payment proof, view billing history, access case files, and communicate through the messaging module.

LEXSHIELD was designed to address common limitations in manual legal-service workflows. In traditional arrangements, client inquiries, case documents, consultation schedules, payment confirmations, and communication records may be scattered across paper files, personal messages, and separate storage folders. The developed system consolidates these records so that authorized users can retrieve information more quickly and perform their tasks with better accountability.

This chapter therefore establishes that the system is more than a static website. It is an integrated case management platform with role-based access, relational database support, file handling, appointment tracking, payment review, and audit logging. These elements work together to improve the organization and monitoring of legal service transactions.

4.2 Overview of the Developed System

LEXSHIELD Web-Based Legal Services and Case Management System

LEXSHIELD is a responsive web application intended for legal offices or legal-service providers that need a practical tool for managing clients, lawyers, cases, payments, appointments, inquiries, and messages. The system can be accessed through a web browser in a local XAMPP environment. It provides an organized interface that separates functions according to user responsibility.

The developed system has three primary user groups. The admin user is responsible for system-level management. The lawyer user focuses on assigned legal work, appointments, messages, profile information, and case-related records. The client user focuses on registration, profile maintenance, lawyer browsing, payment submission, billing, case files, and communication.

The main objective of LEXSHIELD is to provide a centralized digital platform that supports legal service coordination. The system reduces dependence on scattered manual records by storing key information in a database and linking related records through consistent identifiers. This structure helps users trace a client's account, case file, payment proof, appointment, or message without relying on separate manual tracking sheets.

The major objectives of the developed system are as follows:

1. Provide secure login, registration, password recovery, and role-based access for administrators, lawyers, and clients.
2. Allow administrators to manage lawyer and client accounts, review public inquiries, verify manual GCash payments, and configure system settings.
3. Allow lawyers to view their dashboard data, case files, appointments, profile records, client-related information, and messages.
4. Allow clients to register, view lawyers, submit payment proof, check billing and payment history, update their profile, access case files, and book or view appointments.
5. Support digital case-file organization through folders, uploaded documents, document statuses, and assigned lawyer-client relationships.
6. Maintain accountability through audit logs, timestamps, statuses, and review records for sensitive actions such as payment verification.

The system workflow begins when a user opens the website and logs in through the authentication page. After authentication, the system checks the user's role and redirects the user to the correct dashboard. From there, the user can access only the modules allowed for that role. This structure supports security, usability, and operational clarity.

4.3 System Development Methodology

Agile Development Approach

The development of LEXSHIELD followed an Agile software development approach. Agile was selected because the project required progressive refinement of multiple modules, including authentication, dashboards, user management, appointments, payments, case files, messages, and system settings. These components are interconnected, so iterative development made it easier to build, test, and improve each function.

In the requirements-gathering phase, the researchers identified the expected tasks of administrators, lawyers, and clients. The collected requirements emphasized secure access, organized case records, practical payment review, appointment monitoring, communication, and clear account management. These requirements became the basis for the project scope and module list.

During planning, the development team identified the needed technology stack and local deployment environment. The project was placed in the XAMPP htdocs directory, with Apache serving the PHP pages and MySQL storing the database records. The structure also includes optional Node API and Flask application folders for supporting services.

During system design, the researchers organized the application into role-based directories. The admin folder contains modules such as audit logs, inquiries, lawyer management, client management, messages, payments, payment viewing, and system settings. The client folder contains billing, appointments, lawyer browsing, messages, payments, profile, and dashboard pages. The lawyer folder contains appointments, client profile viewing, messages, profile management, and dashboard pages.

During development, individual modules were implemented and connected to shared configuration and bootstrap files. The config folder contains reusable logic for authentication, case files, database connection, storage, messaging, mail, schema handling, and view support. This separation improves maintainability because common operations do not need to be repeatedly written in every page.

During testing, each module was checked for expected behavior. Authentication was tested by confirming that users were redirected to the appropriate dashboard. Payment processing was tested by submitting proof of payment and reviewing it through the admin queue. Messaging was tested through conversation creation, message sending, attachment handling, and receiver visibility. Case files were tested by checking folder organization and document access.

The deployment phase focused on preparing the local environment. The project requires Apache and MySQL through XAMPP, an environment file for database settings, database import through phpMyAdmin, and migration execution through the scripts folder. This process makes the system reproducible for local demonstration, testing, and academic evaluation.

4.4 System Architecture

Three-Tier Client-Server Architecture

LEXSHIELD uses a three-tier client-server architecture composed of the presentation layer, application layer, and data layer. This architecture separates the user interface, business logic, and database storage so that each part of the system has a clear responsibility.

The presentation layer is the browser-based interface used by admins, lawyers, and clients. It is composed of PHP pages, shared layout components, CSS styling, and JavaScript behavior. Users interact with login forms, dashboards, tables, filters, upload controls, profile forms, payment forms, appointment pages, message threads, and administrative panels.

The application layer contains the PHP scripts and shared bootstrap files that process user requests. This layer validates sessions, checks user roles, connects to the database, prepares queries, handles file upload rules, sends or displays messages, loads case files, and applies business rules such as payment status changes or appointment filtering.

The data layer is the MySQL database. It stores accounts, lawyer records, client records, cases, case files, folders, case file documents, appointments, manual payments, reviews, messages, audit logs, inquiries, OTP records, password resets, sessions, notifications, and site settings. Foreign key relationships connect records so the system can trace relationships between users, lawyers, clients, cases, payments, and documents.

The architecture supports maintainability because modifications can be made in the proper layer. For example, visual refinements are usually handled in public CSS or page templates, business logic is handled in PHP and configuration files, and stored information is managed through the SQL schema and migrations.

The architecture also supports security. Users do not directly access the database. Instead, requests pass through authenticated pages and helper functions. Role separation limits the functions shown to each user. File handling is directed through storage-related configuration rather than unrestricted public access.

The workflow can be summarized as follows: a user opens a page, submits a request, the PHP application validates the session and role, the application queries or updates MySQL, and the response is displayed back to the browser. This cycle is repeated across all major modules.

4.5 Hardware and Software Requirements

Resources Needed for Implementation

The successful implementation of LEXSHIELD requires appropriate hardware and software resources. Since the system is web-based and database-driven, the environment must support a local web server, PHP execution, MySQL database management, and browser access.

Table 4.1 presents the recommended hardware specifications.

Component	Minimum Specification	Recommended Specification
Processor	Intel Core i3 or equivalent	Intel Core i5 or higher
Memory (RAM)	4 GB	8 GB or higher
Storage	256 GB available storage	512 GB SSD or higher
Display	1366 x 768 resolution	Full HD display
Internet connection	Required for optional online service	Stable broadband connection
Client device	Desktop, laptop, tablet, or smartpho	Modern browser-capable device

The minimum requirements are sufficient for local demonstration and module testing. The recommended specifications provide better performance when importing the database, uploading files, browsing dashboards, reviewing proof images, and running optional services.

Table 4.2 presents the software requirements.

Software	Purpose
Windows with XAMPP	Local Apache and MySQL server enviro
PHP	Main server-side scripting language
MySQL	Relational database management syste
phpMyAdmin	Database creation and SQL import
Visual Studio Code	Source code editing and project main
Google Chrome or modern browser	Accessing and testing the web interf
Git	Version control and project backup
Node.js	Optional API service support
Python and Flask	Optional supporting application serv

These tools were selected because they are widely used, accessible for student development, and appropriate for a PHP/MySQL capstone project.

4.6 Database Design

Relational Database Structure

The database design of LEXSHIELD is one of the most important parts of the system because it supports organized storage, retrieval, and relationship tracking. The database is named `lexsh_db` and uses the `utf8mb4` character set to support a broad range of text input.

The `users` table stores the core login account information. It contains the user's full name, email, password hash, role, active status, failed login attempts, lockout time, creation date, last login, and avatar stored name. The role field identifies whether the account belongs to an admin, lawyer, or client.

The `lawyers` table stores legal practitioner information connected to a user account. It includes the bar number, specialization, status, biography, background, and contact number. It has a unique relationship with the `users` table, meaning each lawyer profile belongs to one user account.

The `clients` table stores client profile information connected to a user account. It includes contact number, address, and risk level. The risk level supports administrative monitoring by identifying whether a client is low, medium, or high risk according to the system's categorization.

The `cases` table stores formal case records. It includes the case number, title, description, assigned lawyer, client, status, priority, filed date, and closed date. This table links lawyer and client entities, allowing the system to track who is responsible for a case and which client is involved.

The `case_files` table stores digital case-file records. It includes the client user, assigned lawyer user, case identifier, title, description, status, folder name, attachment information, creator, updater, and timestamps. This table supports a document-centered workflow for legal records.

The `case_file_folders` and `case_file_documents` tables support folder organization and file tracking. Documents include original and stored filenames, MIME type, file size, hash, upload status, uploaded by, approval information, rejection reason, confidentiality flag, and timestamps.

Other major tables include `appointments`, `manual_payments`, `lawyer_reviews`, `messages`, `audit_logs`, `quick_inquiries`, `password_resets`, `email_otp`s, `sessions`, `notifications`, and `site_settings`. Together, these tables form the foundation of the complete legal-service workflow.

4.7 Major System Modules

Functional Components of LEXSHIELD

LEXSHIELD is divided into several functional modules. Each module was designed to support a specific operational need while remaining connected to the overall role-based workflow.

The authentication module manages login, registration, logout, password reset, and account access. It ensures that users enter the system through controlled pages and that each user is directed to the correct role-based dashboard.

The admin dashboard module provides centralized management tools. Through this area, the administrator can manage lawyers, manage clients, review inquiries, review payment proofs, view audit logs, update system settings, and send messages to lawyers. This module supports oversight and control of the entire platform.

The lawyer module provides access to lawyer-specific tasks. Lawyers can view dashboard data, manage profile information, access assigned case files, review appointments, view client-related information, and use secure messaging. This helps lawyers focus on active service delivery rather than administrative setup.

The client module provides access to client-specific tasks. Clients can register, log in, browse lawyers, submit manual payment proof, view billing records, manage profile information, access case files, send messages, and view appointments. This gives clients a single portal for common legal-service interactions.

The case-file module organizes legal documents and case-related records. It supports case file titles, identifiers, descriptions, folder names, assigned lawyers, client users, document uploads, document approval statuses, and confidentiality flags. This is important because legal-service systems often depend on accurate record handling.

The payment module supports manual GCash payment submission and admin review. Clients upload proof of payment with payer information, amount, reference number, and notes. Admins review each record and mark it verified or rejected while adding notes when needed.

The messaging module supports secure communication between users. It includes sender and receiver identifiers, message text, optional attachments, encryption flag, sent timestamp, and read status. This supports traceable communication within the platform.

4.8 User Interface Design

Role-Based Navigation and Screens

The user interface of LEXSHIELD was designed to be practical, organized, and role-based. Instead of presenting the same menus to all users, the system displays functions according to the user's responsibility. This reduces confusion and helps users focus on the tasks that apply to them.

The login page serves as the primary access point. After authentication, the system checks the user's role. Admin users are redirected to the admin dashboard, lawyer users to the lawyer dashboard, and client users to the client dashboard. This behavior supports usability and security at the same time.

The admin interface includes management pages for lawyers, clients, inquiries, payments, messages, audit logs, and settings. Tables are used where records need to be searched, filtered, updated, or reviewed. Action buttons allow the admin to add users, update statuses, verify payments, reject payments, or inspect details.

The lawyer interface focuses on case-related service delivery. Lawyers can view assigned information, appointments, messages, and profile details. The design supports repeated daily use by giving lawyers direct access to the information they need to coordinate with clients and manage legal tasks.

The client interface focuses on accessibility and self-service. Clients can browse lawyers, manage profiles, submit payments, check billing, view case files, and communicate through messages. These features reduce the need for repeated manual follow-ups because clients can review their own records through the system.

The public and shared assets, including CSS and JavaScript files, support consistent styling and behavior across the application. The public JavaScript files include behavior for case files, chat, client profile, lawyer browsing, and general site interactions.

Overall, the interface design reflects a role-based management system rather than a static informational website. Its layout supports dashboards, record lists, forms, uploads, filters, and messages that are commonly needed in a legal-service environment.

4.9 Security and Data Protection

Controls Applied in the System

Security is an important concern in LEXSHIELD because the system handles legal-service records, client information, messages, appointments, and payment proof images. The project includes several controls to protect access and support accountability.

User authentication limits access to registered users. Passwords are stored as password hashes rather than plain text. The users table also includes failed login attempt tracking and locked-until fields, which can support account lockout behavior after repeated failed attempts.

Role-based access separates the actions of admins, lawyers, and clients. This prevents a client from using administrative tools and prevents a lawyer from accessing system management functions that are reserved for the admin. Role separation is essential because the three user groups have different levels of authority.

The database uses foreign key constraints to preserve record relationships. For example, lawyer and client profiles are linked to user accounts, cases are linked to lawyers and clients, and messages are linked to sender and receiver accounts. These constraints reduce orphaned records and improve data integrity.

The file-handling structure records document metadata such as original filename, stored filename, MIME type, file size, file hash, upload status, uploader, approval details, rejection reason, and confidentiality flag. This supports better document tracking than uncontrolled file storage.

The manual payment process includes proof file details, reference number, amount, payer information, status, admin notes, reviewer, and review timestamp. This structure makes payment review traceable and helps avoid unclear payment records.

Audit logs support accountability by recording actions, target tables, target identifiers, IP addresses, user agents, and creation timestamps. This is important for administrative monitoring and for investigating changes made within the system.

These security measures do not eliminate every possible risk, but they provide a practical foundation for protecting user access, sensitive records, payment verification, and administrative activity in a local capstone implementation.

4.10 Implementation Process

Local Deployment and Configuration

The implementation of LEXSHIELD was performed in a local development environment using XAMPP. The project folder is placed in the htdocs directory so it can be served through Apache at the local URL for the project.

The first step in deployment is installing and opening XAMPP. Apache and MySQL must be started from the XAMPP control panel. Apache handles the PHP pages, while MySQL stores the `lexsh_db` database used by the application.

The second step is preparing the project folder. The project is placed in `C:\xampp\htdocs\lexs_capstone`. This location allows the system to be accessed through a browser using the local path `http://localhost/lexs_capstone/`.

The third step is creating the environment file. The sample `.env.example` file is copied into `.env`, and the database settings are checked. This file contains configuration values such as the database name, database username, password, application URL, and API URL.

The fourth step is creating and importing the database. The developer opens phpMyAdmin, creates the database, and imports `sql\lexsh_db.sql`. This file contains the schema and seed data needed to initialize the project tables and relationships.

The fifth step is running migrations through the scripts folder. The migration process updates or completes database structures required by the application. This helps ensure that the local database matches the current project files.

The sixth step is opening the website in a browser. The main page can be accessed at `http://localhost/lexs_capstone/`, while the login page can be accessed at `http://localhost/lexs_capstone/auth/login.php`.

Optional services can also be started. The Node API can be run inside the `api` folder using `npm install` and `npm start`. The Flask application can be run inside the `flask_app` folder after activating the virtual environment and installing requirements.

4.11 Testing Strategy

Module, Integration, and User Acceptance Testing

Testing was conducted to determine whether the developed system performs according to its intended functions. The testing activities focused on module behavior, integration between modules, data handling, access control, and user workflows.

Unit testing was applied to individual modules. The login module was tested by entering valid and invalid account credentials. The registration module was tested by creating client accounts and confirming whether the account appeared correctly in the database. The profile module was tested by updating user information and checking whether the changes were saved.

The admin management modules were tested by adding, updating, and deleting lawyer and client records. Search and filter behavior was also checked where applicable. The inquiries module was tested by reviewing public inquiries and changing statuses such as new, read, replied, or closed.

The payment module was tested by submitting a manual GCash payment as a client and reviewing it as an admin. The test checked uploaded proof details, reference number, amount, payment status, admin notes, reviewer data, and review timestamp.

The messaging module was tested by creating conversations, sending messages, attaching files, viewing message lists, and confirming whether recipients could read the messages. This test verified that sender and receiver relationships were properly handled.

The case-file module was tested by checking case file records, folder organization, document metadata, document statuses, assigned lawyer information, client linkage, and available file actions. Because legal documents are sensitive, correct access and organization were considered important during testing.

Integration testing was performed by following end-to-end workflows. For example, an admin creates lawyer and client accounts, a client submits payment proof, the admin verifies the payment, and the client checks billing or payment history. Another workflow involves creating a message from one role and confirming visibility from the receiver's account.

User Acceptance Testing was conducted by checking whether the system's interface and functions were understandable for the intended users. The evaluation focused on whether users could complete core tasks such as logging in, navigating dashboards, managing records, submitting payments, viewing case files, and sending messages.

4.12 Evaluation Results

Functional Performance of the Developed System

The evaluation of LEXSHIELD showed that the system successfully met the major functional requirements identified during development. The role-based structure worked as intended by directing users to the appropriate dashboard and limiting available actions according to user type.

The authentication and account management features supported controlled access. Admin users could manage lawyers and clients, while client users could register and access client-specific pages. This result shows that the system can separate responsibilities among different types of users.

The admin module performed its intended role as the central management area. The administrator could review inquiries, manage lawyer and client accounts, review payment proofs, view audit records, send messages, and update system settings. These results indicate that the system can support administrative supervision.

The lawyer module provided the expected tools for legal-service work. Lawyers could access their dashboard, profile, appointments, messages, and case-related information. This result shows that the system can support lawyer participation without exposing unnecessary administrative features.

The client module successfully supported self-service functions. Clients could manage their profile, browse lawyers, submit payment proof, view billing information, access case files, use messages, and view appointments. This result demonstrates that the system can reduce manual client follow-up by giving clients direct access to relevant records.

The payment review process was evaluated as useful because it organizes proof of payment information in a reviewable queue. The system stores proof filename, amount, payer details, reference number, status, admin notes, reviewer, and review timestamp. This supports transparency and reduces confusion in manual payment handling.

The messaging and case-file modules were also evaluated as important because they support communication and document organization. Legal-service workflows often depend on accurate records and timely communication. By keeping these functions inside the same platform, the system improves coordination among users.

Overall, the evaluation results indicate that LEXSHIELD is functional, practical, and aligned with its goal of supporting legal-service management through a centralized web-based system.

4.13 Discussion

Interpretation of Results

The results of the system development and evaluation show that LEXSHIELD addresses a real need for organized digital support in legal-service operations. The project demonstrates that a PHP/MySQL web application can provide practical tools for managing legal records, client communication, appointments, payments, and administrative oversight.

One significant result is the effectiveness of role-based access. Legal-service environments involve different responsibilities. Administrators need broad management authority, lawyers need access to assigned work and client communication, and clients need access to their own profile, billing, payments, appointments, and case files. The system's separation of roles helps prevent confusion and supports data privacy.

Another significant result is the integration of multiple related workflows. Instead of treating payments, case files, messages, and appointments as separate manual processes, LEXSHIELD connects them within one platform. This makes it easier to monitor the status of client interactions and reduces the need to search across separate records.

The database design also contributes to the usefulness of the system. By linking users, lawyers, clients, cases, case files, payments, messages, and appointments through foreign keys, the system can maintain relationships between records. This improves accuracy and helps avoid disconnected information.

The manual payment feature is especially practical for local implementation because GCash payments are commonly used. The system does not assume fully automated payment gateway integration. Instead, it provides a review workflow that fits manual verification while still recording status, notes, proof details, and timestamps.

The case-file feature is also important because legal work depends on documents. The system's support for folders, document metadata, upload status, approval information, and confidentiality flags provides a foundation for responsible document handling.

However, the system also has limitations. It is primarily configured for local deployment and may require further hardening before public internet hosting. Additional controls such as two-factor authentication, stricter file scanning, automated backups, and production-grade server configuration would improve readiness for real-world deployment.

Despite these limitations, the completed system proves that LEXSHIELD can serve as a functional capstone solution for legal-service management.

4.14 Strengths and Limitations

System Assessment

The developed system has several strengths. First, it centralizes important legal-service workflows in one application. Administrators, lawyers, and clients do not need separate tools for every activity because the system already includes modules for accounts, case files, appointments, messages, payments, inquiries, and settings.

Second, the system uses a relational database with defined relationships. This improves consistency and makes the project more maintainable. Records are connected through user identifiers, lawyer identifiers, client identifiers, case identifiers, and foreign key constraints.

Third, the system supports accountability. Audit logs, timestamps, payment review data, document upload metadata, message read states, and account status fields provide useful traces of activity. These records can help administrators monitor the system and resolve questions about actions taken by users.

Fourth, the system is accessible through a browser and can be run locally through XAMPP. This makes it practical for demonstration, testing, classroom evaluation, and small-scale deployment. The setup process is also clear because the project includes a user manual, SQL schema, migration script, and configuration files.

Fifth, the system is role-based. The separation of admin, lawyer, and client functions improves usability because each user sees the modules most relevant to the user's tasks. It also improves control because system management pages are not exposed to ordinary client users.

The system also has limitations. Since it is configured as a local capstone project, production deployment would require stronger hosting configuration, HTTPS, server-level file protection, backup procedures, and monitoring. The optional API and Flask folders may also require additional setup if they are used in a full deployment.

Another limitation is that payment verification remains manual. This is practical for the current scope, but it still requires admin review and does not automatically confirm payment through a gateway. Future development could integrate a verified payment API if available.

Finally, user evaluation could be expanded. More respondents, longer usage time, and real legal-service scenarios would provide stronger evidence of usability, performance, and effectiveness.

4.15 Summary of Chapter 4

Development Findings

This chapter presented the system development, implementation, testing, evaluation, results, and discussion of LEXSHIELD. The system was described as a web-based legal services and case management platform designed for administrators, lawyers, and clients.

The chapter explained the overview of the developed system, including its objectives, main user roles, and core modules. It also discussed the Agile development methodology used to guide the project from requirements gathering to deployment and maintenance.

The three-tier architecture of the system was described through the presentation layer, application layer, and data layer. This architecture allows the browser-based interface, PHP application logic, and MySQL database to work together while maintaining separation of responsibilities.

The hardware and software requirements were also presented. LEXSHIELD requires a local server environment such as XAMPP, PHP, MySQL, a browser, and supporting development tools. Optional Node.js and Flask services may be used depending on the project's extended configuration.

The database design was discussed in relation to its major tables, including users, lawyers, clients, cases, case files, folders, documents, appointments, manual payments, reviews, messages, inquiries, audit logs, sessions, notifications, and settings. These tables support the main workflows of the system.

The chapter further discussed the major system modules, user interface design, security controls, implementation process, testing strategy, evaluation results, strengths, and limitations. These discussions show that the system is functional and aligned with its intended purpose.

Based on the development and evaluation, LEXSHIELD provides a practical digital platform for organizing legal-service activities. It improves record organization, supports role-based access, enables payment review, assists case-file management, and provides communication tools for users.

The next chapter presents the conclusions drawn from the project and the recommendations for future improvement, implementation, and research.

CHAPTER 5

CONCLUSION AND RECOMMENDATIONS

5.1 Introduction

This chapter presents the conclusion and recommendations based on the development and evaluation of LEXSHIELD. It summarizes the findings of the project, explains whether the objectives were achieved, identifies the impact of the system, and proposes improvements for future development.

LEXSHIELD was created to provide a web-based platform for legal-service management. The project focused on the needs of three user roles: administrators, lawyers, and clients. Its major functions include authentication, account management, lawyer browsing, client management, case files, document organization, appointments, messaging, billing, manual GCash payment review, inquiries, audit logging, and system settings.

The development of the system showed that common legal-service tasks can be organized in a single database-driven application. Instead of relying only on paper records, scattered messages, or separate payment notes, LEXSHIELD allows authorized users to access structured records through role-based dashboards.

5.2 Summary of Findings

The project produced several important findings. First, the system successfully implemented role-based access for admins, lawyers, and clients. This supports security and usability because each user type receives the functions appropriate to the user's responsibility.

Second, the system successfully centralized legal-service records. Lawyer information, client profiles, case records, case-file documents, payment submissions, appointment schedules, messages, inquiries, and audit logs are stored in a relational database. This helps users locate and monitor records more efficiently.

Third, the system supported manual payment verification. Clients can submit GCash payment proof, while admins can review and mark the payment as verified or rejected. This workflow fits the project's scope and improves the documentation of payment activity.

Fourth, the system supported communication and coordination through messages and appointments. These modules are useful because legal-service work often requires continuous interaction between clients and lawyers.

Overall, the findings show that LEXSHIELD achieved its purpose as a functional capstone system for web-based legal-service and case management.

5.3 Conclusion

Assessment of the Completed System

Based on the development and evaluation, LEXSHIELD can be concluded as a viable and useful system for organizing legal-service operations. The system provides a centralized digital environment where administrators can manage users and payments, lawyers can monitor assigned work, and clients can access relevant services.

The system addressed the need for better organization of legal-service records. Through the case-file module, documents and case-related details can be stored with identifiers, folders, upload metadata, statuses, and assigned users. This supports improved record management compared with scattered manual files.

The system also addressed the need for clearer communication. The messaging module allows users to exchange messages within the platform, while the appointment module helps organize consultation-related schedules. These functions support coordination between clients and lawyers.

The payment module addressed the need for documented payment review. Manual payment proof can be submitted, checked, verified, or rejected. The system records payer details, amount, reference number, proof file information, admin notes, reviewer, and review time. This improves transparency in payment handling.

The admin dashboard addressed the need for management oversight. The administrator can monitor lawyers, clients, inquiries, payments, messages, audit logs, and system settings. This makes the admin role more structured and allows better supervision of platform activity.

The use of a relational database strengthened the system by maintaining relationships among users, lawyers, clients, cases, documents, payments, messages, appointments, and other records. This structure makes LEXSHIELD more organized and easier to maintain.

Therefore, the project successfully met its objectives. It demonstrates that a PHP/MySQL web application can be used to create a practical legal-services management system for academic and local implementation.

5.4 Recommendations

Suggested System Improvements

Although LEXSHIELD achieved its main objectives, several improvements are recommended to strengthen future versions of the system.

First, the system should implement two-factor authentication. Since legal-service records may contain sensitive information, an additional verification step would improve account protection beyond username and password authentication.

Second, the system should include stronger production deployment controls. If the application is hosted online, it should use HTTPS, secure server configuration, protected upload directories, regular database backups, error logging, and server monitoring.

Third, the file upload process should be expanded with file scanning, stricter file type validation, size limits, and automated warnings for suspicious files. These improvements would reduce risk when users upload payment proofs, avatars, or legal documents.

Fourth, the payment module may be improved by integrating a verified payment gateway or official payment API. This would reduce manual review time and allow automatic confirmation when payment data can be safely validated.

Fifth, the appointment module may be improved with email or SMS notifications. Reminders for upcoming appointments, pending requests, confirmations, cancellations, and schedule changes would help clients and lawyers avoid missed consultations.

Sixth, reporting features may be added for administrative decision-making. Reports may include payment summaries, active cases, client risk distribution, appointment counts, lawyer workload, inquiry status, and user activity.

Seventh, the system may include document version history. Legal files may change over time, so version tracking would help users identify the latest document while preserving earlier copies for reference.

Eighth, future versions may add a mobile-friendly progressive web application experience. This would improve access for clients and lawyers who frequently use smartphones.

5.5 Impact of the Study

Contribution to Legal-Service Digitalization

The LEXSHIELD project contributes to the field of information technology by demonstrating how a web-based information system can support legal-service management. It shows that common legal-office workflows can be organized through a role-based application using accessible technologies such as PHP, MySQL, JavaScript, CSS, and XAMPP.

For administrators, the system improves oversight by providing tools for managing lawyers, clients, payments, inquiries, messages, and settings. This reduces the burden of maintaining separate manual records and improves the traceability of administrative actions.

For lawyers, the system provides a structured environment for reviewing appointments, case files, profile information, client-related records, and messages. This helps lawyers focus on legal work while still maintaining organized digital access to important information.

For clients, the system improves convenience by allowing account registration, lawyer browsing, profile updates, payment proof submission, billing review, case-file access, appointment viewing, and messaging. These functions support transparency and reduce the need for repeated in-person or manual follow-ups.

For the academic community, the project serves as an example of applying software engineering concepts to a practical legal-services problem. It includes database design, authentication, role-based access, file handling, administrative dashboards, messaging, payment review, and deployment procedures.

The impact of the study is therefore both technical and practical. Technically, it demonstrates the integration of several modules in one web application. Practically, it provides a model for improving organization, communication, and accountability in legal-service workflows.

While the system still requires further hardening for production use, it establishes a strong foundation for future development and real-world adaptation.

5.6 Final Summary

Closing Statement

In summary, LEXSHIELD successfully fulfilled its purpose as a web-based legal services and case management system. The project delivered a role-based platform for administrators, lawyers, and clients, with modules for authentication, dashboards, user management, lawyer browsing, case files, appointments, messaging, billing, manual payments, inquiries, audit logs, and settings.

The system development process showed that Agile methods were suitable for building the project because each module could be developed, tested, and refined incrementally. The final structure of the project reflects organized implementation through separate admin, client, lawyer, authentication, configuration, storage, script, API, and database components.

The testing and evaluation results showed that the system performs the essential functions required by the study. Users can log in according to their roles, access the proper dashboard, manage or view records, submit and review payments, exchange messages, and access case-related information.

The study concludes that LEXSHIELD is a functional and useful solution for improving legal-service organization. It reduces reliance on scattered manual records and provides a central platform for important activities. Its database design, role separation, and workflow modules support more accountable and accessible service management.

The researchers recommend future improvements such as two-factor authentication, production-grade hosting, automated backups, secure file scanning, payment gateway integration, appointment notifications, reporting tools, document version history, and mobile enhancements. These improvements can expand the usefulness and security of the system.

Ultimately, LEXSHIELD demonstrates how information technology can support the modernization of legal-service processes. By combining web development, database management, user access control, file handling, and communication tools, the project provides a meaningful contribution to digital transformation in legal-service management.

This chapter closes the study by affirming that the objectives of the project were achieved and that the system provides a strong foundation for future enhancement, deployment, and research.